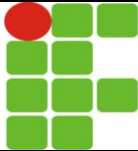


IFGOIANO – CAMPUS URUTAÍ SI		 INSTITUTO FEDERAL GOIANO Campus Urutaí
ED I	3º Matutino	Júnio César de Lima
Lista de Exercícios 4 Listas encadeadas circulares		

Exercício 1: Conversão Vetor-Lista encadeada circular (Importação)

Escreva um método que crie uma lista encadeada circular encadeada circular a partir de um vetor, mantendo a ordem original dos elementos.

Exercício 2: Conversão Lista encadeada circular-Vetor (Exportação)

Escreva um método que copie uma lista encadeada circular para um vetor, mantendo a ordem original dos elementos.

Exercício 3: Concatenação com Preservação (Cópia)

Faça um método para concatenar duas listas encadeada (L1 e L2) em uma nova lista encadeada circular (L3), que deverá ser retornada. Os elementos de L1 devem preceder os de L2. L1 e L2 devem permanecer intactas.

Exercício 4: Remoção em Lote (Validação de Prefixo)

Faça um método para remover os n primeiros elementos de uma lista encadeada circular. O método deve retornar true se a remoção de todos os “n” elementos for concluída com sucesso, ou false caso a lista encadeada circular possua menos de “n” elementos (neste caso, nenhum elemento deve ser removido).

Exercício 5: Busca de Extremos

Faça um método para buscar a posição e o valor do maior elemento da lista encadeada circular, o qual deve ser retornado.

Exercício 6: Permutação de Elementos

Faça um método que receba dois valores (chaves) e realize a troca de posição entre os nós que contêm esses elementos na lista encadeada circular. O método deve validar se ambos os elementos existem e retornar true somente se a troca for concluída com sucesso.

Exercício 7: Filtragem de Dados

Considere uma lista encadeada circular contendo números inteiros positivos. Implemente um método que percorra a lista encadeada circular e retorne a lista encadeada circular

contendo apenas os números pares encontrados. Não usar diretamente o método remove.

Exercício 8: Cálculo Estatístico

Considere uma lista encadeada circular contendo números inteiros positivos. Implemente um método que calcule e retorne a média aritmética de todos os valores presentes na lista encadeada circular. Caso a lista encadeada circular esteja vazia, o método deve tratar a exceção ou retornar zero.

Exercício 9: Verificação de Ordenação

Implemente um método que verifique se os elementos da lista encadeada circular estão organizados de forma lógica. O método deve retornar true se a lista encadeada circular estiver estritamente crescente ou estritamente decrescente, e false caso os elementos estejam desordenados.

Exercício 10: Clonagem de Estrutura

Crie um método que realize a cópia fiel de uma lista encadeada circular L1 para uma nova lista encadeada circular L2. Todos os nós de L2 devem ser instâncias novas (novas alocações de memória), garantindo que alterações futuras em L2 não afetem o conteúdo de L1.

Exercício 11: Cópia com Distinção

Implemente um método que copie os elementos de uma lista encadeada circular L1 para uma nova lista encadeada circular L2, garantindo que L2 não possua elementos repetidos.

Exercício 12: Inversão por Cópia

Escreva um método que receba uma lista encadeada circular L1 e gere uma nova lista encadeada circular L2 contendo os mesmos elementos, porém em ordem inversa.

Exercício 13: Divisão de Lista encadeada circular (Split)

Implemente um método que receba uma lista encadeada circular e um valor n. O método deve localizar a primeira ocorrência do valor n e dividir a estrutura: a partir do nó imediatamente após n, todos os elementos subsequentes devem formar uma nova lista encadeada circular, que será retornada pelo método. Se n não for encontrado ou for o último elemento, retorne uma lista encadeada circular vazia.

Exercício 14: Concatenação com Reuso de Nós (Eficiência)

Escreva um método que receba duas listas encadeadas circulares, L1 e L2, e as concatene em uma única estrutura, retornando o ponteiro para a nova lista encadeada circular.

Exercício 15: Comparação de Estruturas

Implemente um método que verifique se duas listas encadeadas circulares são identicamente iguais. Para serem iguais, elas devem possuir o mesmo número de elementos, na mesma ordem e com o mesmo conteúdo. O método deve retornar um valor booleano.

Exercício 16: Operação de União

Dadas duas listas encadeadas circulares L1 e L2, implemente o método União, que cria uma terceira lista encadeada circular L3 contendo todos os elementos que aparecem em L1 ou em L2. Certifique-se de que L3 não contenha elementos duplicados, mesmo que eles apareçam em ambas as lista encadeadas circulares originais.

Exercício 17: Operação de Intersecção

Dadas duas listas encadeadas circulares L1 e L2 (sem repetidos internamente), implemente o método Intersecção, que cria uma terceira lista encadeada circular L3 contendo apenas os elementos que estão presentes em ambas as listas encadeadas circulares simultaneamente.

Exercício 18: Limpeza de Ocorrências

Em uma lista encadeada circular que permite elementos repetidos, implemente um método que receba um valor X e elimine todas as células que contenham esse valor. O método deve funcionar corretamente mesmo que o elemento X esteja no início, no meio ou no fim da lista encadeada circular.

Exercício 19: Ordenação de Estrutura (Sorting)

Considerando uma lista encadeada circular que armazena números inteiros, escreva um método que ordene a lista encadeada circular em ordem crescente.

Exercício 20: Decomposição por Paridade (Split)

Dada uma lista encadeada circular encadeada circular de números inteiros, escreva um método que a transforme em duas novas listas encadeadas circulares: a primeira contendo apenas os elementos pares e a segunda apenas os ímpares.

Exercício 21: Intercalação por Posição (Merge)

Implemente um método que receba duas listas encadeadas circulares (L1 e L2) e retorne uma terceira lista encadeada circular L3 intercalando os elementos pela posição: 1º de L1, 1º de L2, 2º de L1, 2º de L2, e assim por diante.

Exercício 22: Intercalação Ordenada (Sorted Merge)

Implemente um método que receba duas listas encadeadas circulares já ordenadas e retorne uma nova lista encadeada circular contendo os elementos de ambas, mantendo a ordem crescente no resultado final.

Exercício 23: Contagem Condicional

Implemente um método que receba um valor inteiro n como parâmetro e retorne a quantidade de nós na lista encadeada circular que possuem um valor estritamente maior que n .

Exercício 24: Somatório de Elementos

Escreva um método que percorra toda a lista encadeada circular de inteiros e retorne a soma total de todos os valores contidos nos nós. Caso a lista encadeada circular esteja vazia, o retorno deve ser 0.

Exercício 25: Modelagem de Polígonos (Geometria Computacional)

Um polígono pode ser representado como uma **lista encadeada circular encadeada circular**, onde cada nó armazena as coordenadas (x, y) de um vértice e aponta para o próximo. Para fechar o polígono, o último vértice deve apontar obrigatoriamente para o primeiro.

- **Tarefa:** Projete uma classe Poligono que utilize uma lista encadeada circular encadeada circular interna.
- **Operações:**
 - 1. `inserirVertice(double x, double y)`: Adiciona um novo ponto ao polígono.
 - 2. `removerAresta(int indice)`: Remove o vértice naquela posição, conectando automaticamente o anterior ao próximo.
 - 3. `desenhar()`: Um método que percorra a lista encadeada circular imprimindo as conexões (ex: "Ligando (0,0) a (0,5)").

Exercício 26: Laboratório de Integração de TAD Lista encadeada circular

Escreva um programa principal (*Main*) que utilize o seu TAD Lista encadeada circular para realizar as seguintes operações de manipulação de dados:

- a) **Instanciação:** Crie quatro objetos de lista encadeada circular independentes: L1, L2, L3 e L4.
- b) **Geração de Dados (L1):** Insira sequencialmente na lista encadeada circular L1 dez números inteiros gerados aleatoriamente (no intervalo entre 0 e 99).
- c) **Geração de Dados (L2):** Repita o processo de geração randômica para a lista encadeada circular L2 (também com 10 elementos).
- d) **União de Estruturas:** Concatene as listas encadeadas circulares L1 e L2, armazenando o resultado na lista encadeada circular L3. (Escolha entre a concatenação por cópia ou por reuso, conforme os exercícios anteriores).
- e) **Processamento Inverso:** Popule a lista encadeada circular L4 com todos os elementos presentes em L3, porém dispostos na ordem inversa.
- f) **Auditoria de Dados:** Exiba o conteúdo formatado das quatro listas encadeadas circulares no console para conferência dos resultados.

Exercício 27: Simulação de Strings com listas encadeadas circulares

Considere que uma String pode ser representada como uma lista encadeada circular encadeada circular, onde cada nó contém um único caractere (char). Implemente um conjunto de métodos para manipular essas estruturas.

- a) **`strcreate(char[] str)`:** Converte um vetor de caracteres (char[]) em uma lista encadeada circular encadeada circular. Cada caractere do vetor deve ocupar um

nó da lista encadeada circular. O método deve retornar o ponteiro para o nó de cabeçalho da lista encadeada circular criada.

b) strcnvlc(Lista encadeada circular lista encadeada circular): Operação inversa: converte uma lista encadeada circular encadeada circular de caracteres de volta para um vetor de caracteres (char[] ou String do Java).

c) strvfyl(Lista encadeada circular l1, Lista encadeada circular l2): Determina a **primeira posição** (índice) na lista encadeada circular l1 que contém um caractere que **não existe** em nenhum nó da lista encadeada circular l2. Retorna o índice inteiro ou -1 caso todos os caracteres de l1 estejam presentes em l2.

d) strsbstr(Lista encadeada circular l1, int i1, int i2): Extrai uma **substring** da lista encadeada circular l1. A nova lista encadeada circular deve começar no índice i1 e ter o tamanho i2. O método retorna o cabeçalho da nova lista encadeada circular, mantendo a lista encadeada circular original l1 intacta.

e) strreplace(Lista encadeada circular l1, int pos, int tam, Lista encadeada circular l2): *(Refatorado para clareza)*: Substitui um trecho da lista encadeada circular l1 (que começa em pos e tem comprimento tam) pelo conteúdo integral da lista encadeada circular l2. A operação deve resultar em uma nova lista encadeada circular, sem alterar a l1 original.

f) strcmpi(Lista encadeada circular l1, Lista encadeada circular l2): Compara lexicograficamente (ordem alfabética) duas strings representadas por listas encadeadas circulares. O retorno segue o padrão da função strcmp da linguagem C:

- **-1**: Se a string l1 for menor que l2.
- **0**: Se as strings forem identicamente iguais.
- **1**: Se a string l1 for maior que l2.

Exercício 28: Gestão de Produtos

Considere que o tipo abaixo está armazenado na sua lista encadeada circular. Faça um método para buscar o produto de menor preço. O método deve retornar se a operação foi possível ou não.

Produto
-codigo: int -nomeProd: String -preco: double -qtdeEstoque: int
+Produto() +Produto(codigo:int,nomeProd:String,preco:double,qtdeEstoque:int) +getters&setters()